

FinRelief AI: AI-Powered Debt Relief & Financial Recovery Platform

Project Description:

FinRelief AI harnesses Generative AI to provide intelligent financial analysis and debt recovery strategies, offering users a fast and personalized roadmap out of debt distress. The platform includes a **Financial Analysis Engine**, which calculates real-time Debt-to-Income (DTI) ratios, a **Settlement Strategy Module**, which generates prioritized payment plans, and a **Negotiation Letter Generator**, which delivers professional, hardship, or urgent letters tailored to specific lenders.

Utilizing Google's API with the **Gemini 2.0 Flash** model, FinRelief AI processes user financial profiles to deliver fast, high-quality financial guidance. Built with FastAPI on the backend and React/Vite on the frontend, the platform ensures a seamless, decoupled user experience. With secure API key management via `.env`, JWT authentication, and responsible input handling, FinRelief AI empowers borrowers to take control of their debt with confidence.

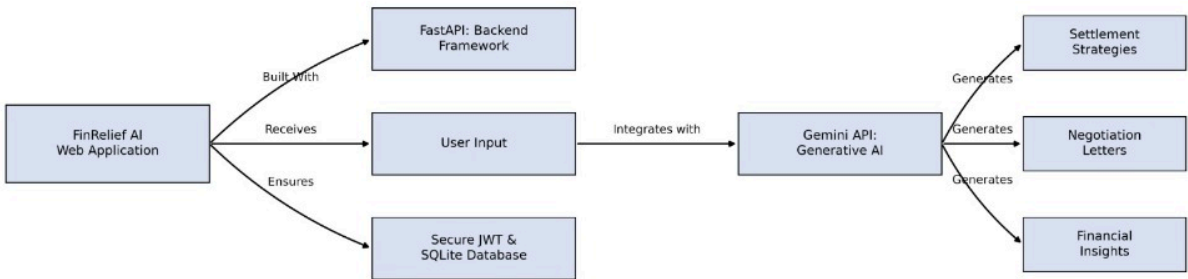
Scenarios

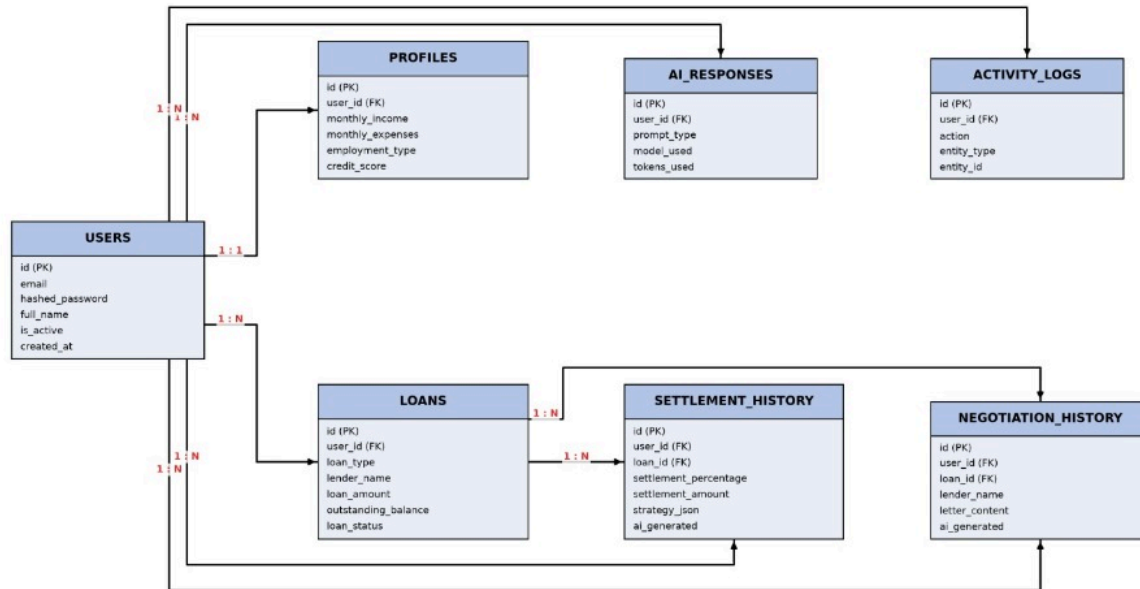
Scenario 1: A user with multiple personal loans and a high Debt-to-Income ratio inputs their financial profile. The system analyzes the data and generates a "Debt Avalanche" strategy, prioritizing the highest-interest loan first, alongside actionable steps to increase their monthly surplus.

Scenario 2: A user who recently defaulted on a credit card needs to negotiate a settlement but cannot afford legal representation. They select the loan and choose a "Professional" tone. FinRelief AI instantly generates a formal, legally-sound settlement offer letter targeting a 45% payoff, ready to be emailed to the bank.

Scenario 3: A borrower facing a sudden medical emergency needs to pause their EMI payments. They select the "Hardship" tone. The AI generates an empathetic, urgent communication to the lender explaining the situation and formally requesting a 3-month forbearance period.

Technical Architecture:





Description: FinRelief AI utilizes a modular three-tier architecture to deliver rapid, AI-driven financial generation. The frontend provides a responsive user interface built with React and CSS, while the FastAPI-based backend orchestrates data validation, database interaction, and prompt construction. It interfaces with the Google Gemini API using the Gemini 2.0 Flash model to generate tailored financial strategies, with secure deployment managed publicly through Render and Vercel.

Pre-requisites:

- **Backend Framework Knowledge:** FastAPI Documentation
- **Google Gemini API Familiarity:** Google AI Studio Documentation
- **Frontend Skills:** React, Vite, and JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Database Management:** SQLite and SQLAlchemy ORM
- **Deployment Hosting:** Vercel (Frontend) and Render (Backend) Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Google Gemini API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Google for rapid JSON data generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the React frontend, FastAPI backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for FastAPI and the Gemini SDK.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Financial Engine Calculator, Settlement Strategy Suggester, and Letter Generator.

- **Activity 2.2:** Implement the FastAPI backend to manage routing, JWT authentication, and user input processing, ensuring smooth API interactions.

Milestone 3: Backend API Development (main.py)

- **Activity 3.1:** Write the main application logic in `main.py` and routers, establishing endpoints for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using React, ensuring a responsive, dark-mode glassmorphism layout.
- **Activity 4.2:** Create dynamic components to display AI results based on user interactions via asynchronous Axios requests.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring CORS and ensuring all database dependencies are correctly installed.
- **Activity 5.2:** Deploy the backend application publicly using Render and the frontend using Vercel over secure public URLs.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Google for our financial data generation needs. This involves researching capabilities to ensure the chosen model aligns well with the application's objectives of generating structured settlement plans and dynamic letters.

Activity 1.1: Generate a Gemini API Key

Before the application can communicate with the Gemini LLM, you need a valid API key.

- **Step 1 — Create a Google AI Studio Account:** Visit <https://aistudio.google.com> and sign in.
- **Step 2 — Create a New API Key:** Click on "Get API Key" and generate a new key for your project.
- **Step 3 — Add the Key to Your Project:** Create a `.env` file in the backend directory:
`GEMINI_API_KEY=your_copied_api_key_here`

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand Requirements:** The application requires strict JSON outputs for financial strategies.
- **Select Optimal Model:** We chose `gemini-2.0-flash` for its extreme generation speed, massive context window, and exceptional ability to adhere to strict JSON schemas.

Activity 1.3: Define the Architecture of the Application

- **Draft Architectural Diagram:** Developed a visual representation mapping the flow from the React UI to the FastAPI backend, down to SQLite and external AI services.
- **Backend Responsibilities:** The FastAPI server handles incoming POST requests, authenticates users, builds structured prompts with financial data, and communicates with Google AI.

Activity 1.4: Set Up the Development Environment

- **Create a Virtual Environment:**

```
python -m venv venv
venv\Scripts\activate
```

- **Install FastAPI & Dependencies:**

```
pip install fastapi uvicorn sqlalchemy passlib python-jose python-d
```

- **Install Google Generative AI SDK:**

```
pip install google-generativeai
```

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

- **Settlement Generator:** Driven by a structured prompt containing the user's loan data, returning a JSON array of prioritized settlement recommendations.
- **Letter Generator:** Combines the loan specifics and selected tone (Professional, Hardship, Urgent) to craft a highly accurate negotiation letter.

Activity 2.2: Implement the FastAPI Backend

- **Process User Input:** Validates JSON payloads containing `loan_id` and `tone`.
 - **Integrate API Calls:** Calls `genai.GenerativeModel.generate_content()` utilizing structured system instructions enforcing JSON-only output.
-

Milestone 3: Backend API Development

Activity 3.1: Writing the Main Application Logic

Define Core Routes:

```
@app.post("/api/settlement/generate")
async def generate_settlement(request: SettlementRequest, db: Session =
    # Retrieve user data
    # Build prompt
    # Call Gemini API
    # Return strategy
```

Integrate Google AI Responses:

```
import google.generativeai as genai
import os

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel('gemini-2.0-flash')

response = model.generate_content(prompt)
```

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

- **Set Up React Structure:** Utilized Vite for rapid scaffolding. Created routing for /dashboard, /settlement, and /letters.
- **Design Responsive Layout:** Applied custom CSS to implement a premium dark glassmorphism aesthetic with animated depth elements.

Activity 4.2: Creating Dynamic Content Rendering

- **Handle Form Submission Asynchronously:** Use `axios.post` to send payload data to the FastAPI backend while displaying a modern loading spinner.
 - **Render Results:** Dynamically inject the AI's JSON response into sleek React components (like the Letter Card and Loan Strategy rows).
-

Milestone 5: Deployment

Activity 5.1: Local Testing and Verification

- **Start the FastAPI Development Server:**

```
uvicorn main:app --reload --port 8000
```

- **Start the React Frontend:**

```
npm run dev
```

- **Verification:** Interact with the frontend at `localhost:5173` to verify CORS policies and successful API communication.

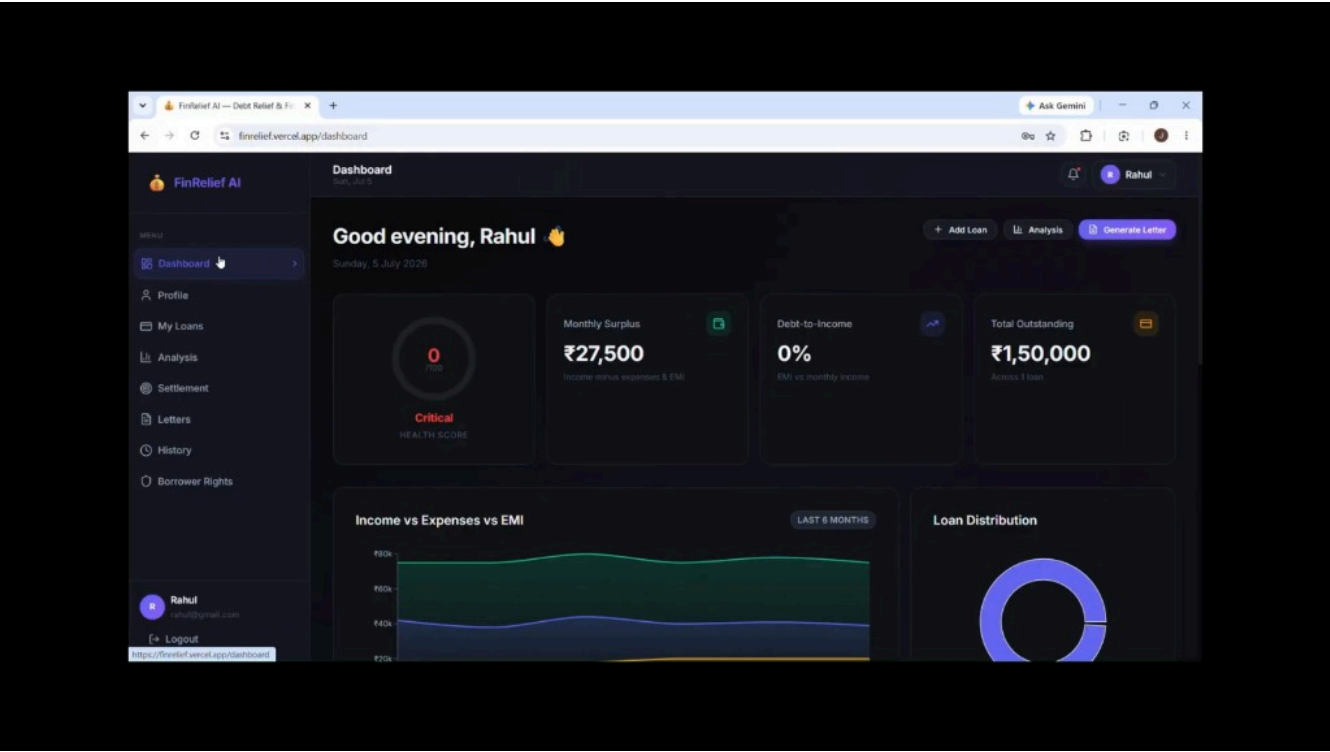
Activity 5.2: Public Deployment

- **Backend:** Deployed the FastAPI server to Render.com, mapping environment variables securely in the Render dashboard.
 - **Frontend:** Deployed the React application to Vercel, pointing API calls to the live Render URL.
-

Exploring the Website's Web Pages:

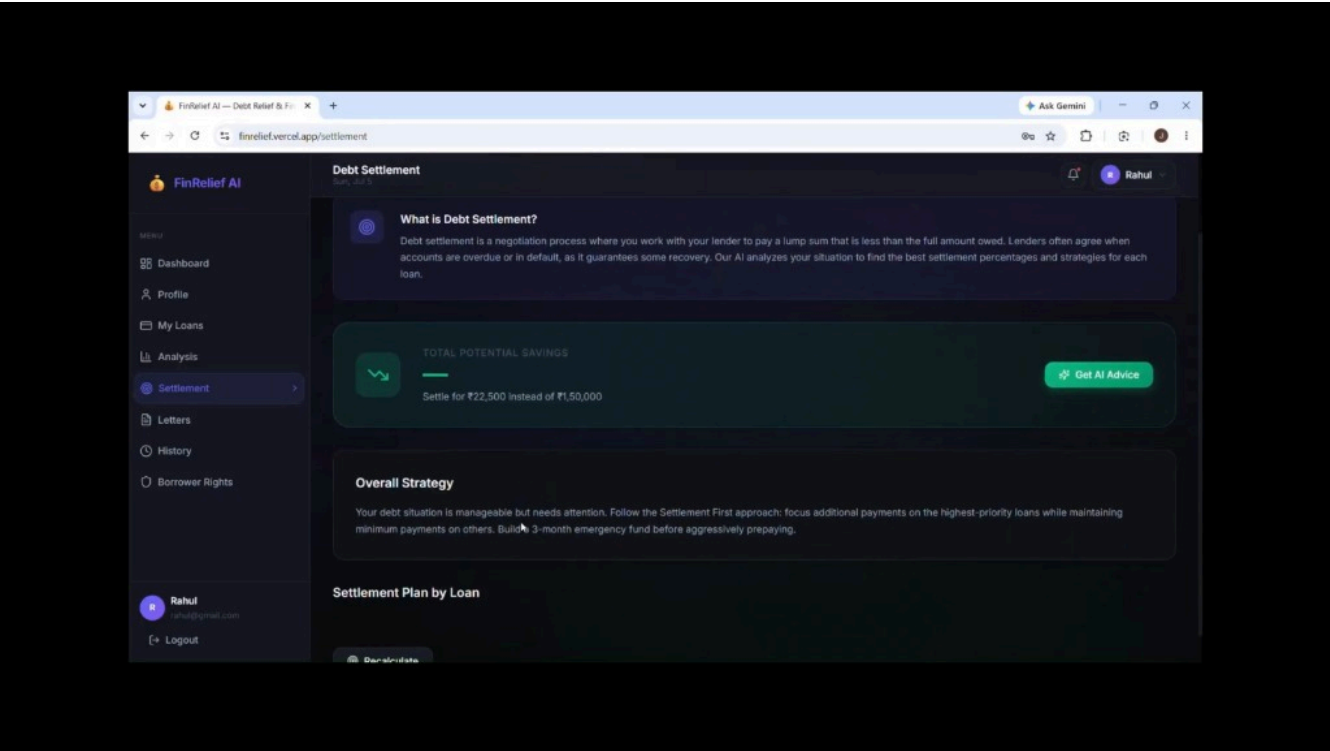
Dashboard Page

Description: The centralized hub for the user, showcasing their holistic financial health. It displays crucial metrics like Monthly Surplus, Debt-to-Income ratio, and Total Outstanding balance across all loans, with an interactive visual distribution chart.



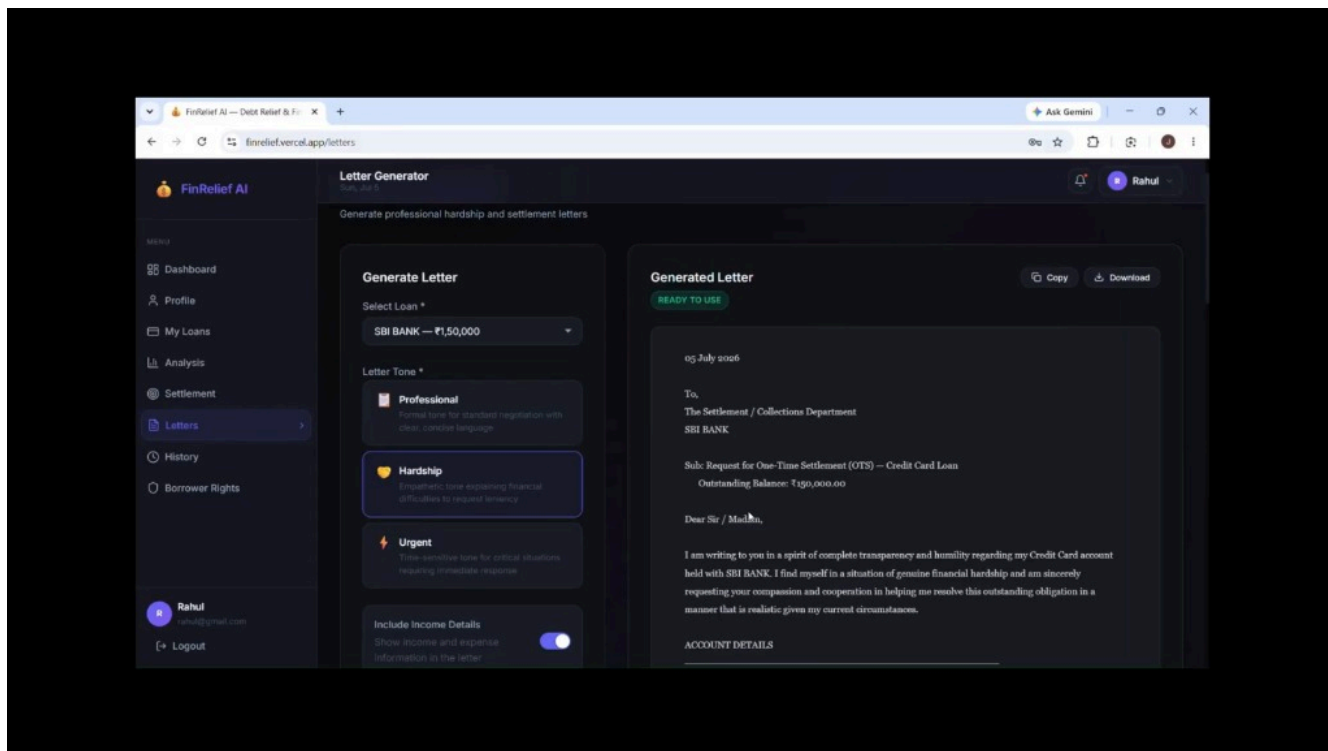
Debt Settlement Page

Description: The Settlement interface dynamically reveals the AI output in structured cards. It displays the **Total Potential Savings** and an **Overall Strategy** paragraph advising the user on prioritization (e.g., Settlement First approach). The viewport clearly outlines exact recommended settlement amounts based on Gemini's real-time financial analysis.



Letter Generator Page

Description: A dedicated interface for generating legally sound communications. Users select their specific loan and target tone (such as **Hardship** or **Professional**). Upon generating, the AI crafts a custom negotiation letter displaying the user's outstanding balance, account details, and a formal request for leniency, ready to be copied or downloaded.



Conclusion:

FinRelief AI is a generative AI platform built with FastAPI and React that streamlines debt management and financial recovery. It uses Google's high-speed Gemini 2.0 Flash model to instantly generate tailored settlement strategies and negotiation letters across professional and hardship tones. The project, which included robust model selection, database development, and cloud deployment via Vercel and Render, highlights how targeted AI and a structured framework can boost financial literacy and provide actionable relief for distressed borrowers. Looking ahead, FinRelief AI offers significant opportunities for expansion, such as integrating banking APIs for real-time data fetching, adding multi-language support, and implementing advanced predictive analytics. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive financial management assistant for borrowers nationwide.